

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «КиСБ»

Тема: «Исследование алгоритмов генерации псевдослучайных последовательностей и методов тестирования чисел на простоту»

Выполнил:
Студент 4-го курса
Группы АС-66
Осовец А. О.
Проверил:
Хацкевич А. С.

Цель работы: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

Ход работы

Вариант 12

1. Генерация и тестирование псевдослучайных последовательностей

В рамках данного этапа необходимо разработать программный модуль генерации последовательностей и провести их первичный анализ.

Требования:

- реализовать алгоритм генерации по варианту в виде программной функции/класса, способной выдавать последовательность произвольной длины;
- подобрать параметры генератора так, чтобы добиться максимального периода последовательности $T_{\max}$ (согласно теоретическим условиям из методички);
- провести базовое статистическое тестирование полученной последовательности (оценить частотность появления элементов, равномерность распределения).

2. Тестирование на простоту и генерация больших простых чисел

На втором этапе сгенерированные на первом этапе псевдослучайные числа интегрируются в модуль поиска и проверки чисел на простоту.

Требования:

- сгенерировать случайное p -битовое число n (используя генератор из Этапа 1);
- установить старший и младший биты в 1 (для гарантии разрядности и нечетности);
- выполнить предварительную фильтрацию (просеивание) делением на малые простые числа (3, 5, 7, ..., 251);
- интегрировать целевой тест из таблицы вариантов для финальной проверки кандидата.

Постановка задачи для варианта 12

Генератор — линейный регистр сдвига с обратной связью (LFSR) степени $m = 5$. Исследуемые характеристические многочлены варианта 12: $f(x) = x^5 + x^2 + 1$, $f(x) = x^5 + 1$, $f(x) = x^5 + x^4 + x^2 + 1$. Начальное состояние

регистра — 10000.

Тест простоты — тест Рабина-Миллера.

Теоретические положения

Для LFSR степени m максимальный период ненулевой последовательности равен $T_{\max} = 2^m - 1$. Он достигается тогда и только тогда, когда характеристический многочлен примитивен над $GF(2)$. Для $m = 5$ максимальный период равен 31.

Многочлены $x^5 + 1$ и $x^5 + x^4 + x^2 + 1$ имеют чётное число ненулевых коэффициентов, поэтому $x = 1$ является их корнем над $GF(2)$; такие многочлены приводимы и не могут быть примитивными.

Тест Рабина-Миллера представляет $n - 1 = d \cdot 2^s$, где d нечётно, и для каждого основания a проверяет, что $a^d \equiv 1 \pmod{n}$ или $a^{d \cdot 2^r} \equiv -1 \pmod{n}$ для некоторого $0 \leq r < s$. Используемый набор из семи оснований $\{2, 325, 9375, 28178, 450775, 9780504, 1795265022\}$ даёт детерминированный результат для всех 64-битных беззнаковых чисел.

Особенности реализации

Программа написана на C++17 и не использует динамических контейнеров и выделения памяти в куче. Состояние LFSR хранится в одном байте; бит обратной связи вычисляется как чётность (state AND taps).

Модульное умножение и возведение в степень выполняются фиксированным числом итераций (64) с выбором результата через битовую маску (select_u64), без условных переходов внутри арифметики и без 128-битного целого типа.

Так как пятибитовый регистр имеет лишь 31 состояние, при формировании каждого нового кандидата выход LFSR смешивается со счётчиком Вейля (умножение номера кандидата на 0x9E3779B97F4A7C15). Это исключает повторение одних и тех же r -битных слов. Решето делением на простые числа 3...251 выполняется полностью, без досрочного выхода.

Листинг программы (main.cpp)

```
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

namespace {

constexpr uint8_t kRegisterMask = 0x1FU;
constexpr uint32_t kStatisticsBits = 1024U;

struct Lfsr {
    uint8_t state;
    uint8_t taps;

    static uint8_t parity(uint8_t value) {
        value ^= static_cast<uint8_t>(value >> 4U);
        value ^= static_cast<uint8_t>(value >> 2U);
        value ^= static_cast<uint8_t>(value >> 1U);
    }
};
```

```

        return static_cast<uint8_t>(value & 1U);
    }

    uint8_t next_bit() {
        const uint8_t result = static_cast<uint8_t>(state & 1U);
        const uint8_t feedback = parity(static_cast<uint8_t>(state & taps));
        state = static_cast<uint8_t>((state >> 1U) | (feedback << 4U));
        return result;
    }
};

struct PolynomialCase {
    const char* polynomial;
    uint8_t taps;
};

constexpr PolynomialCase kPolynomials[] = {
    {"x^5 + x^2 + 1", 0x05U},
    {"x^5 + 1", 0x01U},
    {"x^5 + x^4 + x^2 + 1", 0x15U},
};

constexpr uint16_t kSmallPrimes[] = {
    3U, 5U, 7U, 11U, 13U, 17U, 19U, 23U, 29U, 31U, 37U, 41U,
    43U, 47U, 53U, 59U, 61U, 67U, 71U, 73U, 79U, 83U, 89U, 97U,
    101U, 103U, 107U, 109U, 113U, 127U, 131U, 137U, 139U, 149U,
    151U, 157U, 163U, 167U, 173U, 179U, 181U, 191U, 193U, 197U,
    199U, 211U, 223U, 227U, 229U, 233U, 239U, 241U, 251U
};

constexpr uint64_t kMillerRabinBases[] = {
    2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL
};

struct PrimalityTestCase {
    uint64_t number;
    bool expected_prime;
};

constexpr PrimalityTestCase kPrimalityTestCases[] = {
    {2ULL, true}, {97ULL, true}, {1000000007ULL, true},
    {1ULL, false}, {341ULL, false}, {561ULL, false},
    {1105ULL, false}, {341550071728321ULL, false}
};

uint32_t period(uint8_t taps, uint8_t seed) {
    Lfsr generator{ seed, taps };
    const uint8_t initial = generator.state;
    uint32_t value = 0U;
    do {
        static_cast<void>(generator.next_bit());
        ++value;
    } while (generator.state != initial && value <= 31U);
    return value;
}

void print_statistics(uint8_t taps, uint8_t seed) {
    Lfsr generator{ seed, taps };
    uint32_t zeroes = 0U;
    uint32_t ones = 0U;
    for (uint32_t i = 0U; i < kStatisticsBits; ++i) {
        if (generator.next_bit() == 0U) {
            ++zeroes;
        }
        else {
            ++ones;
        }
    }
    const double expected = static_cast<double>(kStatisticsBits) / 2.0;
    const double chi_square =
        ((static_cast<double>(zeroes) - expected) * (static_cast<double>(zeroes) - expected) +
         (static_cast<double>(ones) - expected) * (static_cast<double>(ones) - expected)) / expected;
    printf("%u bit: 0 -> %u, 1 -> %u, chi^2 = %.6f\n",
           kStatisticsBits, zeroes, ones, chi_square);
}

uint64_t select_u64(uint64_t when_zero, uint64_t when_one, uint64_t bit) {
    const uint64_t mask = 0ULL - (bit & 1ULL);
    return (when_zero & ~mask) | (when_one & mask);
}

uint64_t add_mod(uint64_t left, uint64_t right, uint64_t modulus) {
    const uint64_t sum = left + right;
    const uint64_t reduced = sum - modulus;
    const uint64_t overflow = static_cast<uint64_t>(sum < left);

```

```

    const uint64_t at_least_modulus = static_cast<uint64_t>(sum >= modulus);
    return select_u64(sum, reduced, overflow | at_least_modulus);
}

uint64_t multiply_mod(uint64_t left, uint64_t right, uint64_t modulus) {
    uint64_t result = 0ULL;
    left %= modulus;
    for (unsigned int i = 0U; i < 64U; ++i) {
        const uint64_t candidate = add_mod(result, left, modulus);
        result = select_u64(result, candidate, right & 1ULL);
        left = add_mod(left, left, modulus);
        right >>= 1U;
    }
    return result;
}

uint64_t power_mod(uint64_t base, uint64_t exponent, uint64_t modulus) {
    uint64_t result = 1ULL % modulus;
    base %= modulus;
    for (unsigned int i = 0U; i < 64U; ++i) {
        const uint64_t product = multiply_mod(result, base, modulus);
        result = select_u64(result, product, exponent & 1ULL);
        base = multiply_mod(base, base, modulus);
        exponent >>= 1U;
    }
    return result;
}

bool miller_rabin_64(uint64_t number) {
    if (number < 2ULL) {
        return false;
    }
    if ((number & 1ULL) == 0ULL) {
        return number == 2ULL;
    }

    uint64_t odd_part = number - 1ULL;
    unsigned int twos = 0U;
    while ((odd_part & 1ULL) == 0ULL) {
        odd_part >>= 1U;
        ++twos;
    }

    for (uint64_t base : kMillerRabinBases) {
        if ((base % number) == 0ULL) {
            continue;
        }
        uint64_t value = power_mod(base, odd_part, number);
        if (value == 1ULL || value == number - 1ULL) {
            continue;
        }
        bool witness_passed = false;
        for (unsigned int r = 1U; r < twos; ++r) {
            value = multiply_mod(value, value, number);
            if (value == number - 1ULL) {
                witness_passed = true;
            }
        }
        if (!witness_passed) {
            return false;
        }
    }
    return true;
}

uint64_t lfsr_word(Lfsr& generator, unsigned int bits) {
    uint64_t value = 0ULL;
    for (unsigned int i = 0U; i < bits; ++i) {
        value = (value << 1U) | static_cast<uint64_t>(generator.next_bit());
    }
    return value;
}

bool removed_by_small_prime(uint64_t candidate) {
    bool composite = false;
    for (uint16_t prime : kSmallPrimes) {
        composite = composite || ((candidate % prime) == 0ULL && candidate != prime);
    }
    return composite;
}

struct PrimeSearchResult {
    uint64_t prime;
    uint32_t candidates;
    uint32_t removed_by_sieve;
};

```

```

    uint32_t removed_by_miller_rabin;
};

PrimeSearchResult find_prime(Lfsr& generator, unsigned int bits) {
    PrimeSearchResult result{ 0ULL, 0U, 0U, 0U };
    const uint64_t bit_mask = (1ULL << bits) - 1ULL;
    for (;;) {
        const uint64_t stream_word = lfsr_word(generator, bits);
        const uint64_t weyl_word =
            static_cast<uint64_t>(result.candidates) * 0x9E3779B97F4A7C15ULL;
        uint64_t candidate = (stream_word ^ weyl_word) & bit_mask;
        candidate |= (1ULL << (bits - 1U));
        candidate |= 1ULL;
        ++result.candidates;
        if (removed_by_small_prime(candidate)) {
            ++result.removed_by_sieve;
            continue;
        }
        if (!miller_rabin_64(candidate)) {
            ++result.removed_by_miller_rabin;
            continue;
        }
        result.prime = candidate;
        return result;
    }
}

bool run_primality_self_checks() {
    bool all_passed = true;
    for (const PrimalityTestCase& item : kPrimalityTestCases) {
        const bool actual = miller_rabin_64(item.number);
        const bool passed = actual == item.expected_prime;
        printf("n = %" PRIu64 " : %s\n", item.number,
            actual ? "prime" : "composite", passed ? "" : " (ERROR)");
        all_passed = all_passed && passed;
    }
    return all_passed;
}

void print_usage(const char* program) {
    printf("Usage: %s [bit_length 8..63]\n", program);
}

} // namespace

int main(int argc, char* argv[]) {
    unsigned int bit_length = 40U;
    if (argc == 2) {
        const unsigned long parsed = strtoul(argv[1], nullptr, 10);
        if (parsed < 8UL || parsed > 63UL) {
            print_usage(argv[0]);
            return 1;
        }
        bit_length = static_cast<unsigned int>(parsed);
    }
    else if (argc > 2) {
        print_usage(argv[0]);
        return 1;
    }

    constexpr uint8_t seed = 0x10U; // Non-zero state: 10000.
    printf("Laboratory work 1, variant 12\n");
    printf("LFSR seed: 10000; maximum possible period for degree 5: 31\n\n");

    for (const PolynomialCase& item : kPolynomials) {
        const uint32_t observed_period = period(item.taps, seed);
        printf("f(x) = %s period = %u; %s\n", item.polynomial, observed_period,
            observed_period == 31U ? "primitive" : "not primitive");
        print_statistics(item.taps, seed);
    }

    printf("\nMiller-Rabin self-checks\n");
    const bool self_checks_passed = run_primality_self_checks();
    printf("self-check result: %s\n", self_checks_passed ? "PASS" : "FAIL");

    printf("\nPrime generation (%u bit candidates)\n", bit_length);
    Lfsr generator{ seed, kPolynomials[0].taps };
    const PrimeSearchResult result = find_prime(generator, bit_length);
    printf("prime = %" PRIu64 " (0x%" PRIx64 ")\n", result.prime, result.prime);
    printf("candidates: %u; sieve rejected: %u; Miller-Rabin rejected: %u\n",
        result.candidates, result.removed_by_sieve, result.removed_by_miller_rabin);
    printf("Miller-Rabin bases are deterministic for all unsigned 64-bit inputs.\n");
    return 0;
}

```

Результаты работы программы

Сборка и запуск: `clang++ -std=c++17 -O2 main.cpp -o kish_lab1_variant12`; аргумент командной строки — разрядность кандидата (8...63, по умолчанию 40).

```
$ ./kish_lab1_variant12
Laboratory work 1, variant 12
LFSR seed: 10000; maximum possible period for degree 5: 31

f(x) = x^5 + x^2 + 1      period = 31; primitive
1024 bit: 0 -> 496, 1 -> 528, chi^2 = 1.000000
f(x) = x^5 + 1           period = 5; not primitive
1024 bit: 0 -> 820, 1 -> 204, chi^2 = 370.562500
f(x) = x^5 + x^4 + x^2 + 1 period = 15; not primitive
1024 bit: 0 -> 548, 1 -> 476, chi^2 = 5.062500

Miller-Rabin self-checks
n = 2: prime
n = 97: prime
n = 1000000007: prime
n = 1: composite
n = 341: composite
n = 561: composite
n = 1105: composite
n = 341550071728321: composite
self-check result: PASS

Prime generation (40 bit candidates)
prime = 1002184885789 (0xE956DFC61D)
candidates: 3; sieve rejected: 2; Miller-Rabin rejected: 0
Miller-Rabin bases are deterministic for all unsigned 64-bit inputs.
```

Рисунок 1 — Вывод программы

Таблица 1 — Характеристики LFSR (1024 бита, начальное состояние 10000)

Многочлен $f(x)$	Период Т	Примитивный	Нулей	Единиц	χ^2
$x^5 + x^2 + 1$	31	да	496	528	1,0000
$x^5 + 1$	5	нет	820	204	370,5625
$x^5 + x^4 + x^2 + 1$	15	нет	548	476	5,0625

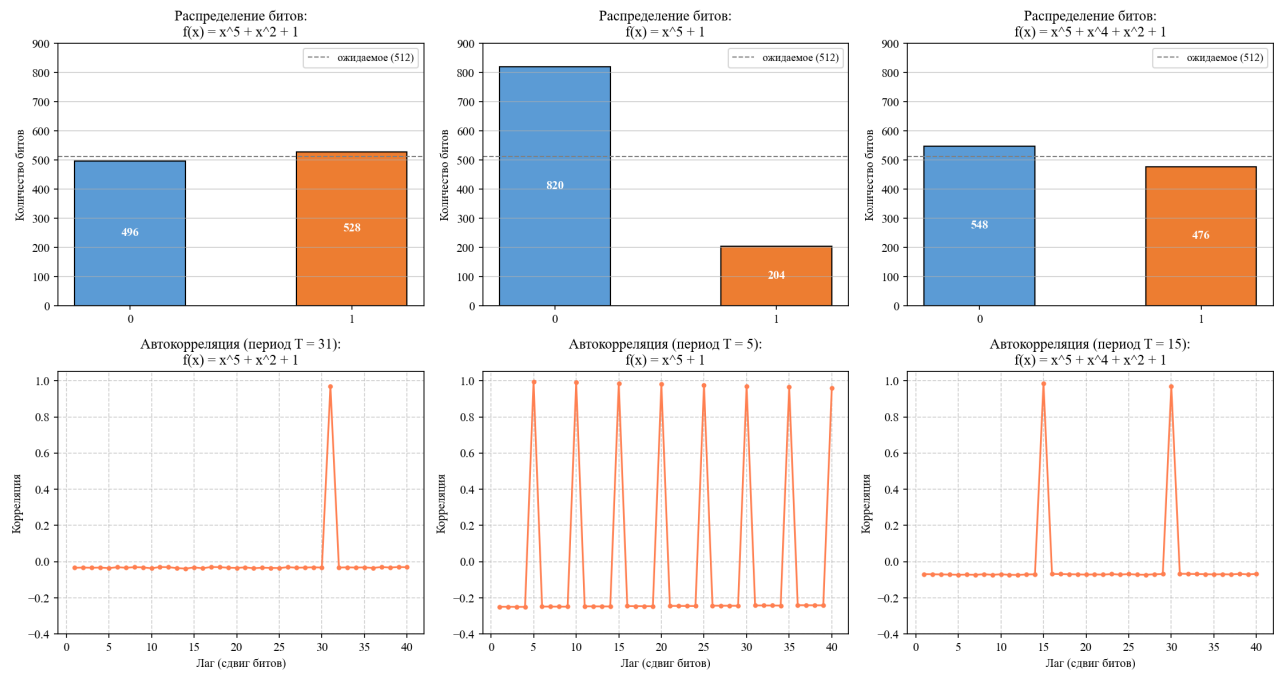


Рисунок 2 — Распределение битов и автокорреляция последовательностей LFSR

Для примитивного многочлена автокорреляция вне пиков постоянна и равна $-1/31$, пик повторяется с периодом 31. У непримитивных многочленов пики появляются на лагах, кратных 5 и 15 соответственно.

Таблица 2 — Журнал генерации простых чисел (время — медиана 30 запусков программы, включая запуск процесса; сам поиск занимает доли миллисекунды)

Бит	Найденное простое	Кандидатов	Решето	Рабин-Миллер	Время, мс
8	137	1	0	0	2,56
16	60133	3	2	0	1,94
24	9004999	1	0	0	1,96
32	2519223389	16	13	2	2,02
40	1002184885789	3	2	0	1,95
48	164742859290469	7	6	0	2,02
56	55440571841860687	13	10	2	2,04
63	5135519329536055547	18	9	8	2,14

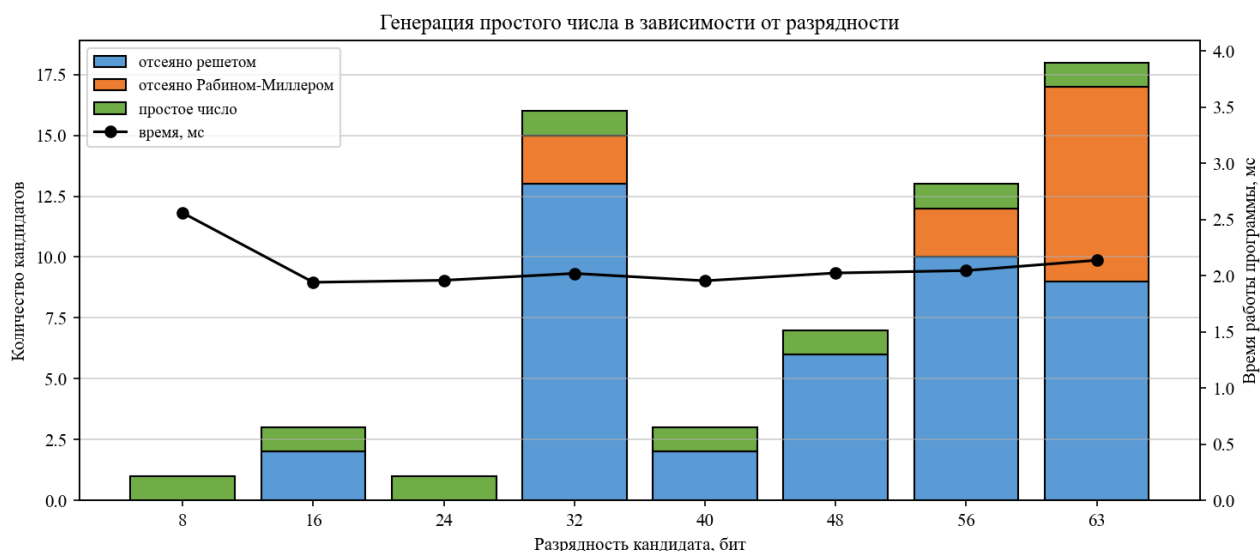


Рисунок 3 — Количество отсеянных кандидатов и время генерации

Вывод: в ходе работы освоены программная реализация и практическое применение алгоритмов генерации псевдослучайных последовательностей, методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел. Установлено, что из трёх многочленов варианта 12 примитивным является только $x^5 + x^2 + 1$: он даёт максимальный период $T = 31$ и близкое к равномерному распределение битов ($\chi^2 = 1,0$). Непримитивные многочлены дают периоды 5 и 15 и заметный перекоп распределения ($\chi^2 = 370,6$ и $5,1$). На основе примитивного LFSR реализована генерация простых чисел разрядностью от 8 до 63 бит с просеиванием и тестом Рабина-Миллера; контрольные проверки, включая числа Кармайкла 561 и 1105, пройдены успешно.